504 HTTP Status Code in ASP.NET Core Web API

Back to ASP.NET Core Web API Tutorial

504 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss how to use the Gateway Timeout HTTP Status Code in an ASP.NET Core Web API application and implement it on an Linux server and on how to Return the HE Service Unavailable HTTP Status Code in an ASP.NET Core Web API application with Examples.

What is HTTP Status Code?

The Hot HTTP Status Code, also known as the "Gateway Timeout" error, indicates that a server while acting as a gateway or proxy has received no response from an upstream server it needed to access in order to complete the request. The upstream server that the gateway server is either down or not responding in time. The root cause or status variables where the server is down or not responding in time are beyond the scope of this article.

504 HTTP Status Code in ASP.NET Core Web API:

In the context of an ASP.NET Core Web API, a 504 Status Code typically indicates that the API server, acting as a gateway or proxy, has not received a response from an upstream server in a timely manner. This error can arise in various scenarios, such as when the upstream server is overloaded, slow, or unreachable, or when there is a network issue or misconfiguration. Here are some common reasons for a 504 Gateway Timeout error in ASP.NET Core Web API:

Common Reasons for a 504 HTTP Status Code in ASP.NET Core Web API:

The following are some of the Common Reasons for a 504 HTTP Status Code in ASP.NET Core Web API:

- **Upstream Server Timeout:** If an ASP.NET Core Web API application acts another server (e.g., database, external API, or microservice) and the upstream server does not respond in a timely manner, it may result in a 504 Gateway Timeout error. The upstream server may be busy or unresponsive.
- **Network Issues:** Network connectivity issues between the ASP.NET Core application and the upstream service can lead to a 504 Gateway Timeout error. Network latency or disruptions can cause this error.
- **Resource Limitations:** If the upstream server is overloaded with requests or under heavy load, it may fail to respond promptly, causing a timeout. That means the upstream server has insufficient resources (CPU, memory, bandwidth) to handle incoming requests in a timely manner, which can lead to slow response times and ultimately a 504 Gateway Timeout error.

How to Return 504 HTTP Status Code in ASP.NET Core Web API:

In ASP.NET Core Web API, we can return a 504 HTTP Status Code in several ways, such as either a controller action, middleware, or exception handling.

Returning a 504 Status Code Directly:

The simplest method involves returning a 504 Status Code directly from a controller action by using the StatusCode method of the ControllerBase class. This approach is suitable for scenarios where you want to explicitly indicate a timeout. The following code is authorized, or where the upstream server is unreachable.

```
Controller.cs
namespace Your.Namespace.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ExampleController : ControllerBase
    {
        private readonly HttpClient _httpClient;
        private readonly ILogger<ExampleController> _logger;

        public ExampleController(HttpClient httpClient,
                                 ILogger<ExampleController> logger)
        {
            _httpClient = httpClient;
            _logger = logger;
        }

        // Endpoint that calls an upstream service
        [HttpGet("data")]
        public IActionResult GetData()
        {
            // Simulating a timeout scenario
            var timeoutExceptionMessage = "The upstream server timed out";
            _logger.LogWarning("Timeout occurred: {Message}", timeoutExceptionMessage);

            // Return 504 Gateway Timeout
            return StatusCode(504, new { Message = timeoutExceptionMessage });
        }
    }
}
```

Using Custom Middleware to Handle 504 Gateway Timeout Errors in ASP.NET Core Web API:

If you want to apply the timeout globally, consider implementing a custom middleware component. Middleware inspects and modifies HTTP requests and responses. The following code demonstrates a simple middleware to handle timeouts:

```
Middleware/GatewayTimeoutMiddleware.cs
namespace Your.Namespace.Middleware
{
    public class GatewayTimeoutException : Exception
    {
        public GatewayTimeoutException(string message) : base(message)
        {
        }
    }

    public class GatewayTimeoutMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly ILogger<GatewayTimeoutMiddleware> _logger;

        public GatewayTimeoutMiddleware(RequestDelegate next,
                                        ILogger<GatewayTimeoutMiddleware> logger)
        {
            _next = next;
            _logger = logger;
        }

        public Task Invoke(HttpContext context)
        {
            // Simulating a timeout scenario
            var timeoutException = new GatewayTimeoutException("The upstream server timed out");
            _logger.LogWarning("Timeout occurred: {Message}", timeoutException.Message);

            // Set the response status code to 504 Gateway Timeout
            context.Response.StatusCode = 504;
            context.Response.ContentType = "application/json";

            // Write the response body with the error message
            var errorResponse = new { Message = timeoutException.Message };
            return context.Response.WriteAsJsonAsync(errorResponse);
        }
    }

    public static class GatewayTimeoutMiddlewareExtensions
    {
        public static IApplicationBuilder UseGatewayTimeoutMiddleware(this IApplicationBuilder app)
        {
            return app.UseMiddleware<GatewayTimeoutMiddleware>();
        }
    }
}
```

Register the Middleware:

Then register this middleware in your Program.cs file:

```
Program.cs
using Your.Namespace.Middleware;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();

var app = builder.Build();

// Use the custom gateway timeout middleware
app.UseGatewayTimeoutMiddleware();

// Map controllers
app.MapControllers();

app.Run();
```

Modifying the Sample Controller:

Now, modify the Sample Controller as follows:

```
Controller.cs
namespace Your.Namespace.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ExampleController : ControllerBase
    {
        private readonly HttpClient _httpClient;
        private readonly ILogger<ExampleController> _logger;

        public ExampleController(HttpClient httpClient,
                                 ILogger<ExampleController> logger)
        {
            _httpClient = httpClient;
            _logger = logger;
        }

        // Endpoint that calls an upstream service
        [HttpGet("data")]
        public IActionResult GetData()
        {
            // Simulating a timeout scenario
            var timeoutExceptionMessage = "The upstream server timed out";
            _logger.LogWarning("Timeout occurred: {Message}", timeoutExceptionMessage);

            // Return 504 Gateway Timeout
            return StatusCode(504, new { Message = timeoutExceptionMessage });
        }
    }
}
```

Response:

```
Body   Cookies   Headers (4)   Test Results        504 Gateway Timeout   347 ms   215 B
{
    "Message": "The upstream server timed out"
}
```

How to Fix Troubleshoot a 504 HTTP Status Code in ASP.NET Core Web API?

Troubleshooting a 504 error in an ASP.NET Core environment, please follow the below steps:

- **Check the Upstream Server:** Verify if the upstream server is up and running. Check its health and status. Look for any errors or performance issues on the server.
- **Network Diagnostics:** Use network troubleshooting tools to ensure ASP.NET Core application can connect to the upstream server. Check for DNS issues, firewall rules, or network bottlenecks.
- **Timeout Settings:** Adjust timeout settings in your ASP.NET Core application if necessary. Ensure that the timeout settings are appropriate for the expected response times from the upstream server.
- **Monitor Performance:** Use monitoring tools to track the performance of your upstream service. Ensure that the server can handle the expected load without slowing down.
- **Load Balancing:** Implement load balancing if you have multiple instances of the upstream service. This can help distribute the load and prevent timeouts.
- **Logs and Error Handling:** Enable detailed logging in your ASP.NET Core application. Configure appropriate error handling to log 504 errors and provide more information.
- **Scale Upstream:** Scale up the resources of your upstream server if it's under heavy load. Scale out by adding more instances to handle the load.

In the next article, I will discuss Model Binding Techniques in ASP.NET Core Web API with Examples. In this article, I hope you enjoy this article **504 Gateway Timeout HTTP Status Code in ASP.NET Core Web API** with examples. I hope you enjoy the article.

Previous Article
HTTP Status Code in ASP.NET Core Web API

Next Article
Model Binding in ASP.NET Core Web API

2 thoughts on "504 HTTP Status Code in ASP.NET Core Web API"

Jayden
January 2, 2025 at 1:12 am

PLEAS you upload Web Api Tutorial exactly. Please let me know.

Reply

Jaya Krishna Namburu
January 2, 2025 at 1:14 am

Yes. Now, we are updating the series.

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name * Email * Website

Post Comment

© Dot Net Tutorials | Website Designed by Suresh Pasal